

Iluminación



Temario

Tema 1: Introducción: Revisión histórica da computación gráfica

Tema 2: Estándares gráficos Evolución histórica, OpenGL vs DirectX

Tema 3: Formas 2D e Antialiasing (final del curso)

Tema 4: Transformacións xeométricas, 2D, 3D,

Tema 5: Proyeccións, modelo de cámara sintética.

Tema 6: Modelado e texturas

Tema 7: Color, Iluminación y sombreado

Tema 8: Determinación de superficies visibles, z-Buffer

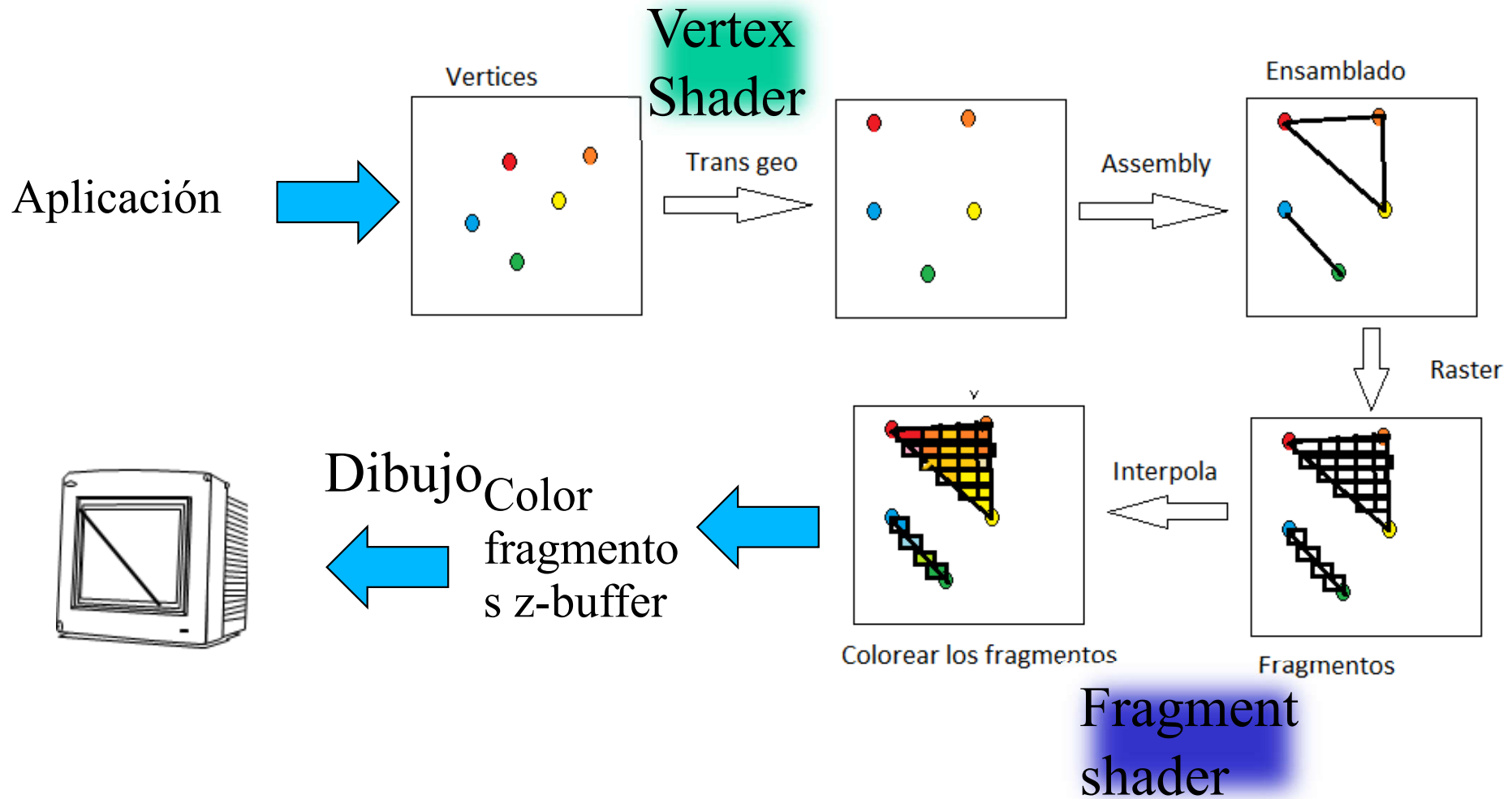
Luces en OpenGL

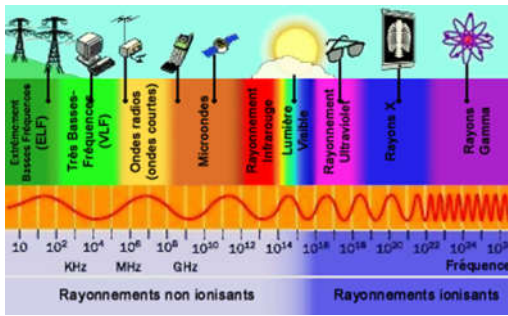
- Tema 7: Color. Iluminación e sombreado
 - Bases físicas da iluminación.
 - Bases físicas del color.
 - Modelo de Goureaux, Modelo de Phong

Bibliografía

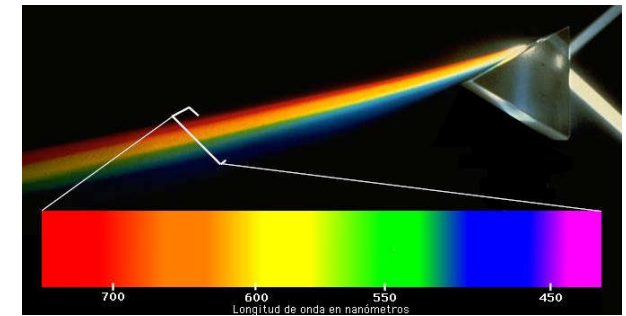
- GRÁFICOS POR COMPUTADORA CON OPENGL, Hearn, Donald; Baker, Pauline.
 - Capítulo 10 pp 576-596, pp 659-668.
- Redbook 8.0 o versión superior.
- LearnOpenGL.com

Pipe 3.3





Que es la Luz



La luz es una **onda electromagnética** que es percibida por el ser humano y cuya frecuencia o suma de frecuencias determina su color.

La luz se caracteriza por tres propiedades físicas

1. Frecuencia y/o Longitud de Onda. Estas dos propiedades son inversamente proporcionales y definen el **color** de la luz.

–**Frecuencia:** Es el número de ciclos de la onda que pasan por un punto en un segundo (Hz).

–**Longitud de Onda:** Es la distancia física entre dos crestas sucesivas de la onda. En el espectro visible, se mide habitualmente en nanómetros (nm).

2. Intensidad (Amplitud). Determina el **brillo** o la cantidad de energía que transporta la onda. Una mayor amplitud se traduce en una luz más intensa.

3. Pureza o Saturación (Espectro). Raramente percibimos una única frecuencia pura (luz monocromática). Lo que vemos suele ser una **suma de frecuencias**.

–**Luz Monocromática:** Una sola frecuencia (ej. un láser).

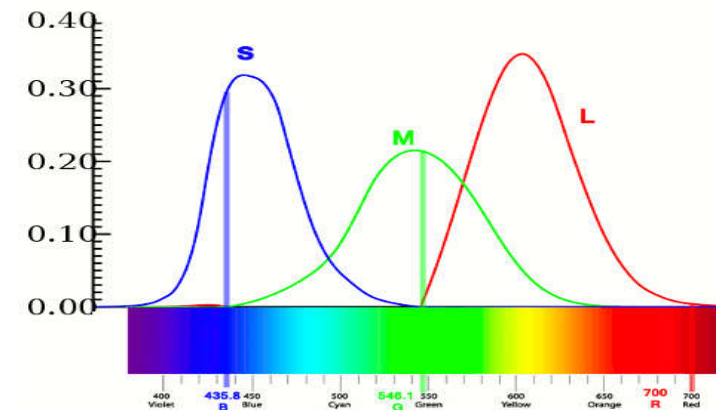
–**Luz Policromática:** Una mezcla de muchas frecuencias (ej. la luz solar). El conjunto de estas frecuencias es lo que llamamos **espectro**

Caracterización humana de la luz

- Dos tipos fundamentales de detectores, los bastones y los conos.
 - **Bastones:** son células fotorreceptoras especializadas en la **visión nocturna** y en condiciones de baja luminosidad, **no distinguen colores**. Tienen su máxima sensibilidad en los 507 nm (luz verde).
 - **Conos:** son células fotorreceptoras responsables de la **visión diurna** y la **percepción del color**: Existen **tres tipos de conos**, cada uno sensible a diferentes longitudes de onda; **rojos** (longitudes de onda largas), **verdes** (longitudes de onda medias), **azules** (longitudes de onda cortas) Los conos-S (azules) son más sensibles a la luz que los conos-L (rojos) y conos-M (verdes).

- El espectro de la intensidad luminosa será:

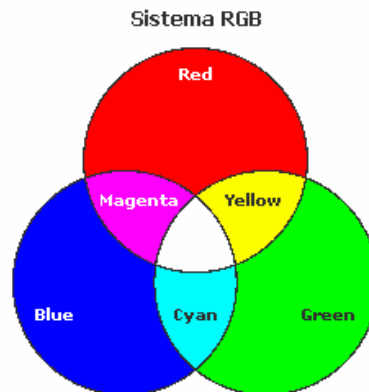
$$I(\lambda) = R.r(\lambda) + V.v(\lambda) + A.a(\lambda)$$



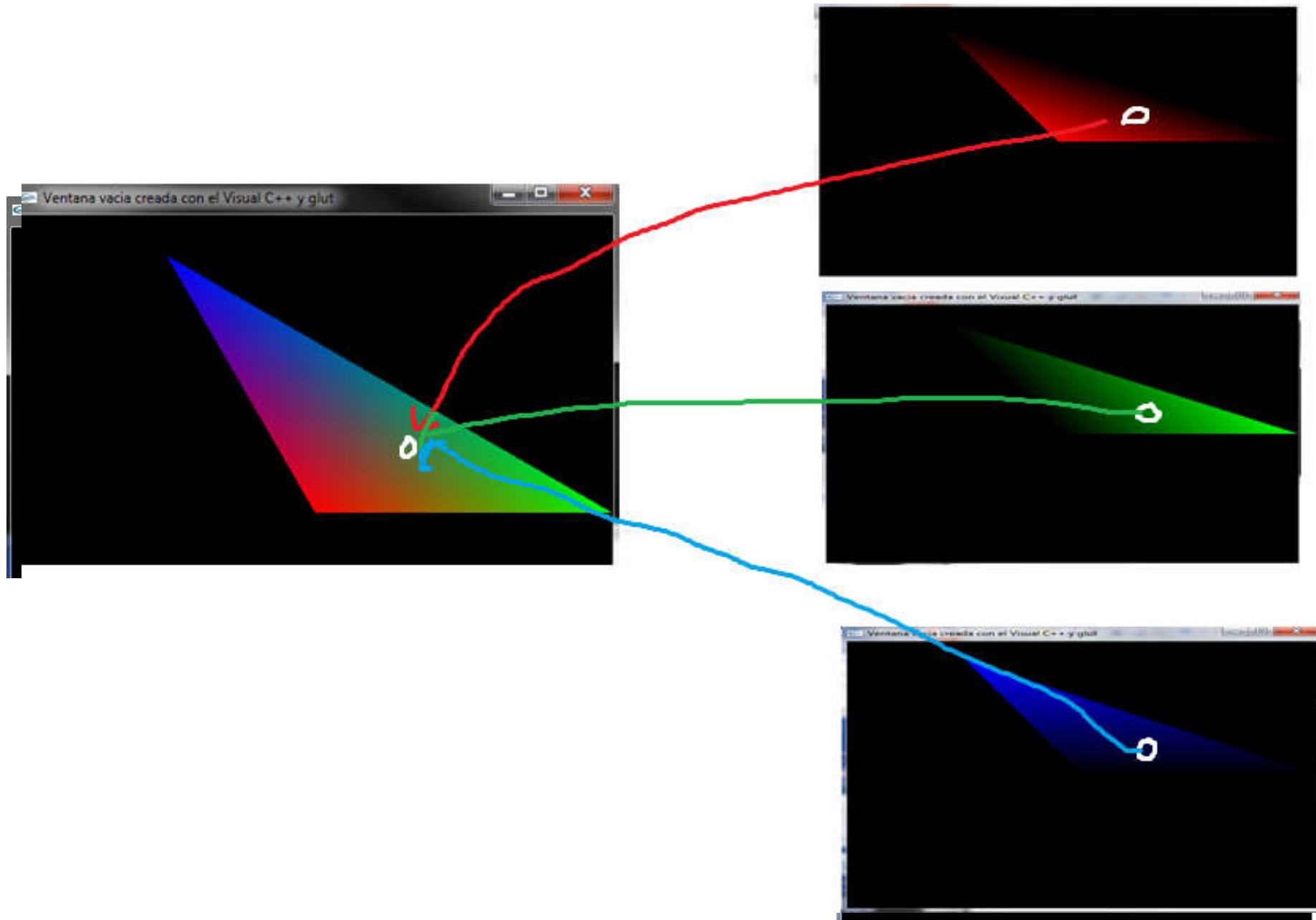
Por esta razón, en **las imágenes de síntesis** se caracterizar un color por medio de sus **componentes R, G, B** (Red, Green, Blue). Sin embargo, existen modelos más precisos que realizan una discretización mucho más fina del espectro luminoso.

Los colores aditivos

- La teoría de los **colores aditivos** se basa en la combinación de luz para crear diferentes colores. Los **tres colores** primarios aditivos son el **rojo, el verde y el azul** (RGB). Esta teoría explica cómo se generan los colores mediante la suma de estos componentes de luz:
 - La combinación de dos colores primarios en proporciones iguales produce colores secundarios; Rojo + Verde = Amarillo, verde + Azul = Cian, Rojo + Azul = Magenta⁴
 - La mezcla de **los tres colores primarios** en intensidades máximas produce **luz blanca**.
 - La **ausencia total** de estos colores resulta en **negro**.
- **OpenGL**, utiliza el modelo de color aditivo RGB para representar colores en pantalla. OpenGL permite especificar colores utilizando valores para los componentes rojo, verde y azul, en un rango de 0 a 1.
- La **síntesis aditiva** es usada OpenGL porque los **monitores y pantallas**, funcionan **emitiendo luz** en estos tres colores primarios para crear la imagen final.

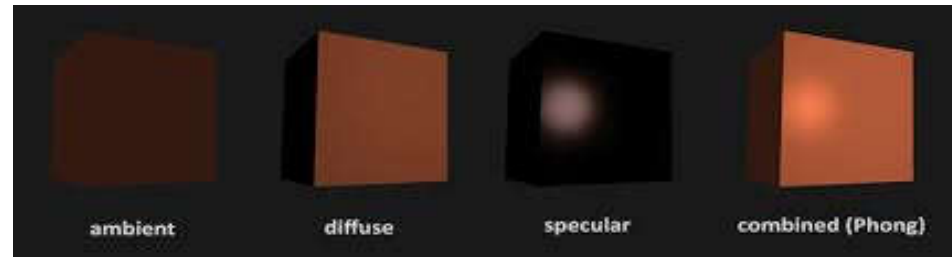


Interpolación del color

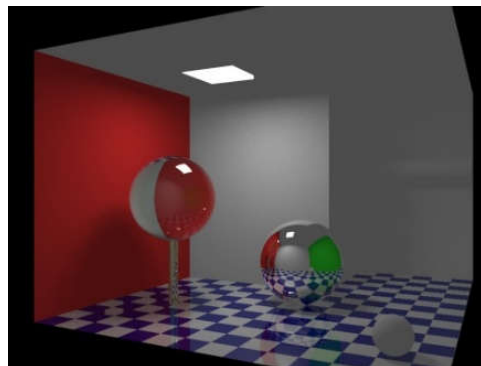


Modelo de Iluminación.

- Hay dos tipos principales de modelos de iluminación.
 - **Locales.** que se utilizan en aplicaciones donde prima la **velocidad de renderizado**, sólo considerando las **características inmediatas del punto** y su entorno cercano, sin tener en cuenta las interacciones complejas de la luz con otros objetos de la escena. El más clásico es el **Modelo de Phong**.



- **Globales.** Consideran la **escena en conjunto**, suelen ser lentos, aunque sus resultados **mucho mejores**, prima la **calidad**, considera las **interacciones complejas** de la luz en toda la escena, incluyendo los efectos indirectos entre objetos. Los más conocidos son los de **radiosidad y trazado de rayos**.



Modelo local de iluminación.

- Utiliza un modelo de **iluminación local** en el cual los cálculos se realizan en función de:
 - **Fuentes de luz**
 - Posición: Localizada o infinitamente alejada.
 - Intensidad y Color de la fuente.
 - Número: Número de luces.
 - Distribución lumínica: uniforme, focalizada, direccional, etc.
 - Geometría: puntual, esférica, lineal, etc.
 - **Objetos**:
 - Distancias: al observador y a la fuente de luz
 - Material/Propiedades ópticas: transparencia, refracción, etc.
 - Cromaticidad: Color superficial propio.
 - Interacción: **No existe interacción lumínica** con otros objetos de la escena.
 - **Observador**.-Dirección de observación.
 - **Simplificaciones iniciales**
 - Fuentes puntuales.
 - Objetos opacos.
 - No hay inter-reflexiones

Modelo de iluminación simple

- Modelo de iluminación

$$I = f(p, pv, \{MGMO\}, \{MGCL\})$$

p : Punto de Cálculo de la iluminación.

pv : Posición del punto de vista del observador

$\{MGMO\}$: Modelo geométrico - material de los objetos

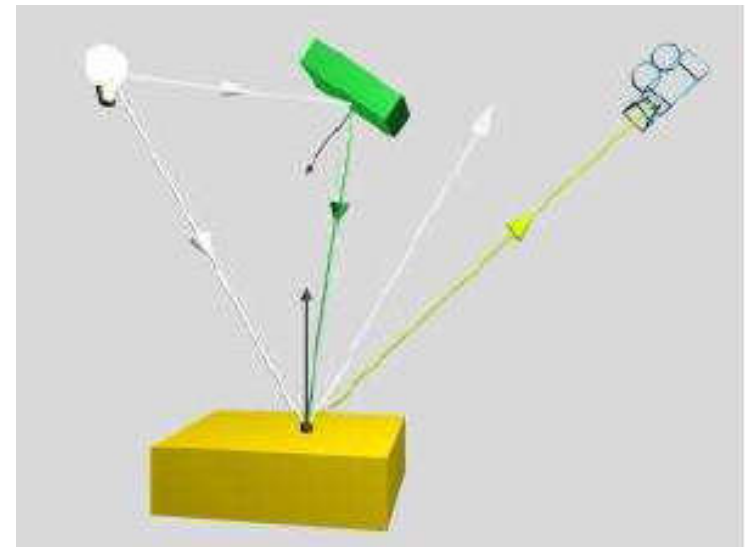
$\{MGCL\}$: Modelo geométrico - cromático de la luz

I : Intensidad Luminosa observada en el punto p



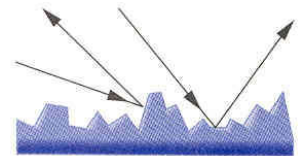
Simplificaciones iniciales

- Fuentes puntuales.
- Objetos opacos.
- No hay inter-reflexiones.



Modelo de Phong (1973)

- Es un **modelo empírico** de iluminación local.
- Fuentes puntuales.
- Los objetos (no fuentes) no emiten su propia luz.
- La luz está formada por:
 - **Luz ambiental:** Luz **no direccional**, no identificable. Solo depende **de su color y del objeto**. Su variación implica cambios uniformes en la iluminación de los objetos. No genera sombras ni matices de color.
 - **Luz Lamber o difusa:** Está **asociada a un foco de luz**. Su variación está asociada a **cambios de intensidad o posición del foco**. Genera sombras y matices de color en función del ángulo con el que incide la luz en la superficie.
 - **Luz especular o de Phong.** Está asociada a un **foco intenso** de luz. Se aprecia en objetos con **brillo**, el caso extremo es **un espejo**. Su variación está asociada a cambios de intensidad, posición del foco, y punto de vista.
 - **Luz Emisiva** la que da apariencia de que objeto **emita luz**.

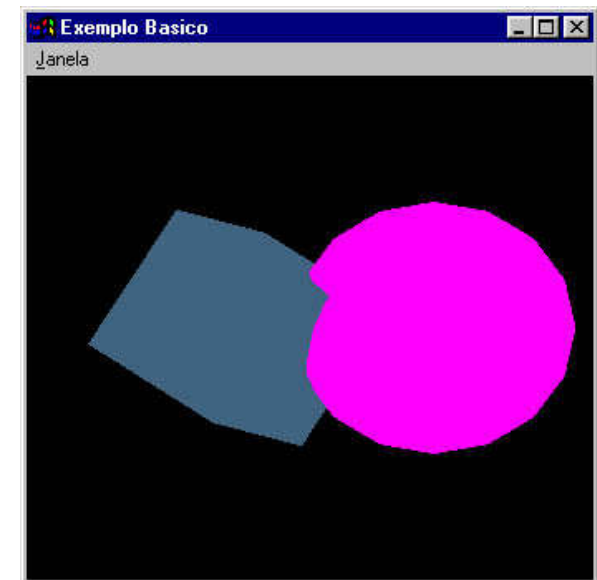


La luz ambiental (modelo empírico)

- La luz ambiental es aquella que **no proviene de una dirección concreta**, incide sobre todas las partes del objeto por igual.

$$I = I_a \cdot K_a$$

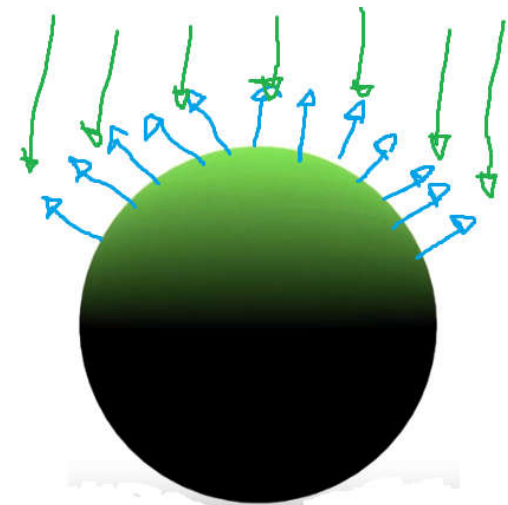
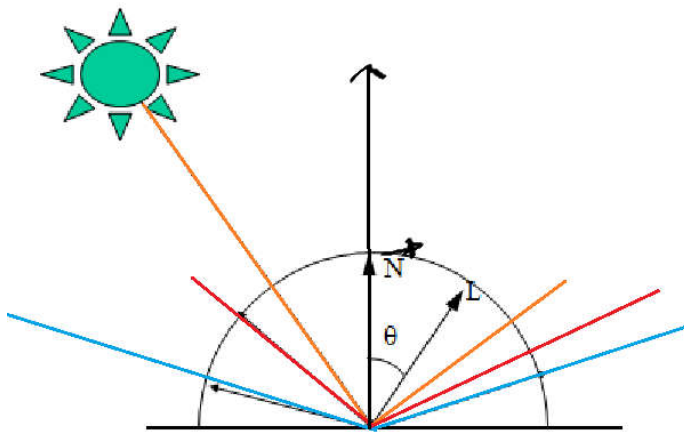
- Donde
 - k_a *coeficiente de reflexión ambiental* $[0,1]$ *dependiente de cada objeto.*
 - I_a *intensidad ambiental en todo punto del espacio.*
- La expresión modela todo lo dicho de la luz ambiente.
- Muestra los objetos **como siluetas**.



La luz difusa (Lambert)

- La luz difusa es aquella que **proviene** de una **dirección particular** y se refleja en todas direcciones, siguiendo la **ley de Lambert**. Este tipo de luz afecta principalmente a las partes del objeto donde la luz incide directamente.
- La luz difusa es reflejada por la **superficie** de un objeto de **manera uniforme** en todas direcciones, lo que significa que **no** depende del **ángulo de observación**, sino **únicamente** del **ángulo de incidencia de la luz**. Este comportamiento se rige por la **ley de Lambert**, que establece que la **intensidad de la luz difusa reflejada es proporcional al coseno del ángulo entre la dirección de la luz incidente y la normal a la superficie**.

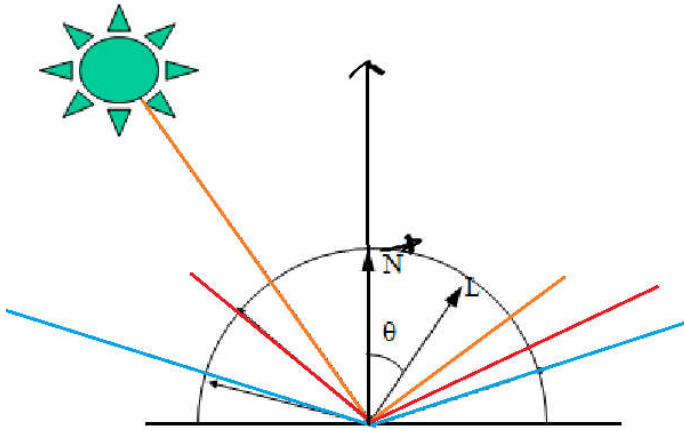
$$I = I_L \cdot K_d \cos(\theta) = I_L \cdot K_d \cdot (\vec{N} \bullet \vec{L})$$



La luz difusa (Lambert)

- Para que esta ecuación sea efectiva se debe verificar que la **normal de la cara del objeto esté normalizado**.

$$I = I_L \cdot K_d \cos(\theta) = I_L \cdot K_d \cdot (\vec{N} \bullet \vec{L})$$



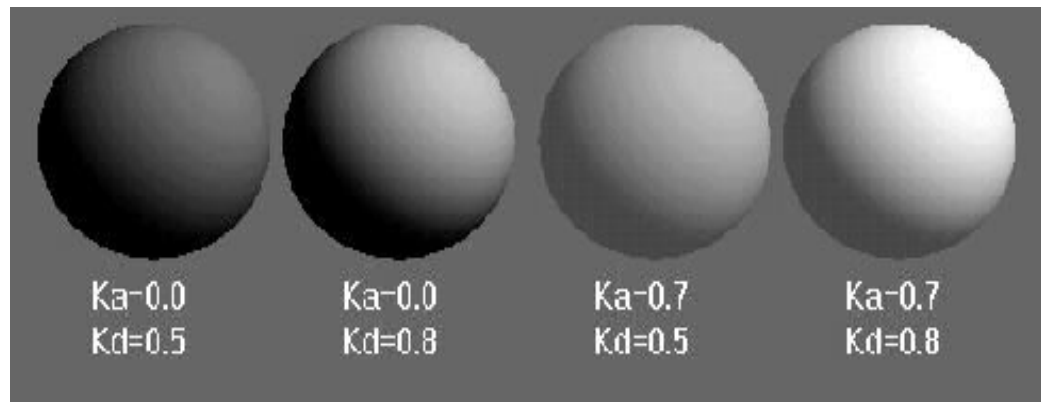
I_L : Intensidad de la fuente

θ : Ángulo de incidencia $[0^\circ, 90^\circ]$

K_d : Coeficiente de reflexión para la luz difusa $[0,1]$

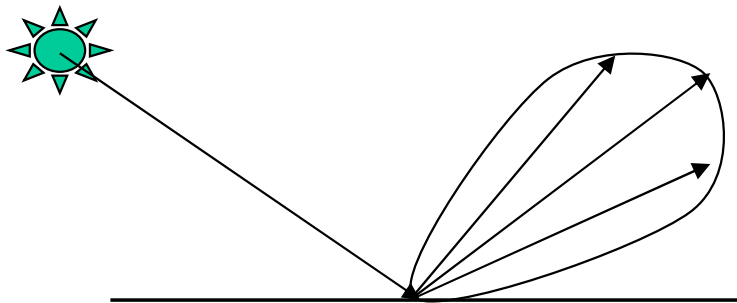
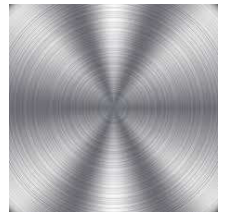
$\vec{N}$: Normal a la superficie (unitario)

$\vec{L}$: Vector de iluminación (unitario)



La luz especular (blinn-phong)

- La luz especular procede de **una dirección** concreta depende de la **posición del observador** y las **propiedades del objeto**.
 - Está asociada a objetos **brillantes o pulidos**.
 - **Las reflexiones especulares** suelen ser del **mismo color** que la **fente de luz**.
 - Caso ideal: Espejo, la luz se refleja exactamente en el ángulo incidente.
 - **Depende** del ángulo de la **luz incidente**.
 - Depende de la **posición del observador**.
 - En superficies imperfectas la luz se refleja dentro de un patrón tridimensional.
 - En **superficies perfectas** rayo **incidente y reflejado coplanares**.
 - En superficies imperfectas patrón de reflexión. Hay luz fuera del plano y el ángulo ideal de reflexión.



La luz especular

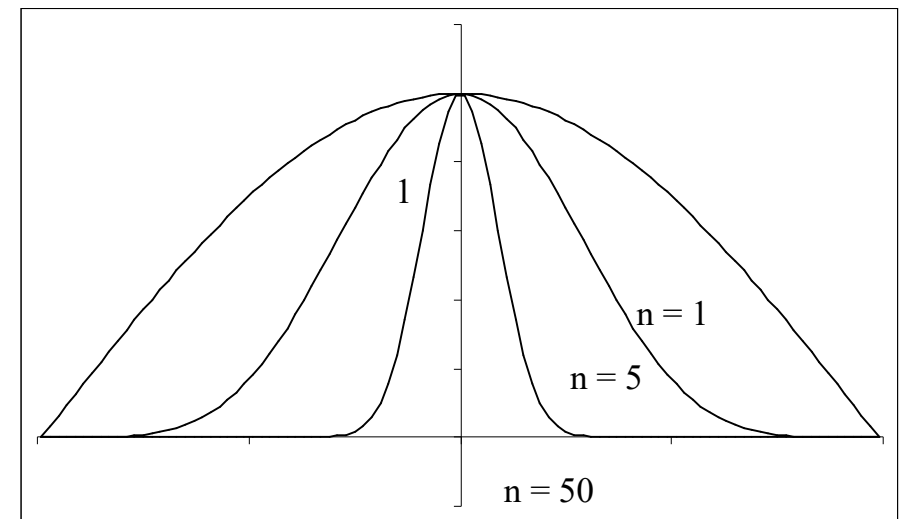
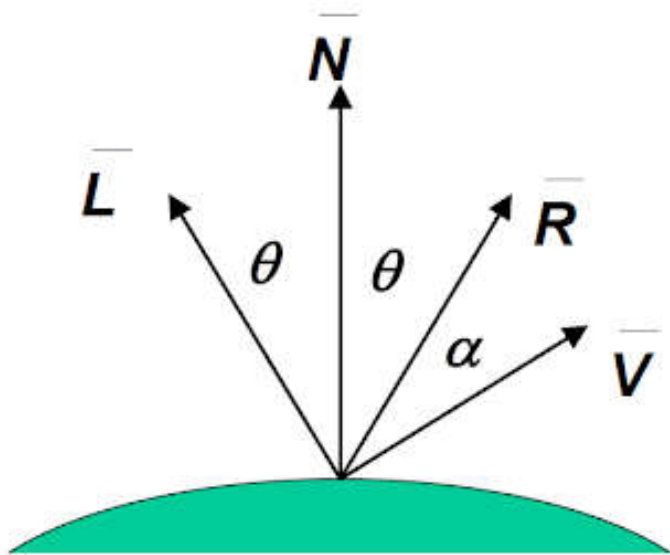
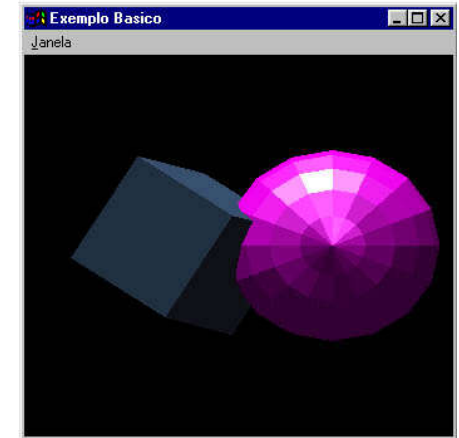
- Modelo de iluminación Especular:

$$I = I_L \cdot K_S \cos^n(\alpha) = I_L \cdot K_S \cdot (\vec{R} \cdot \vec{V})^n$$

α : Ángulo entre $\vec{R}$ y $\vec{V}$. Angulo entre vista y reflexión.

K_S : Coeficiente de reflexión especular dependiendo del material del objeto $[0,1]$

n : Coeficiente de especularidad o concentración del brill



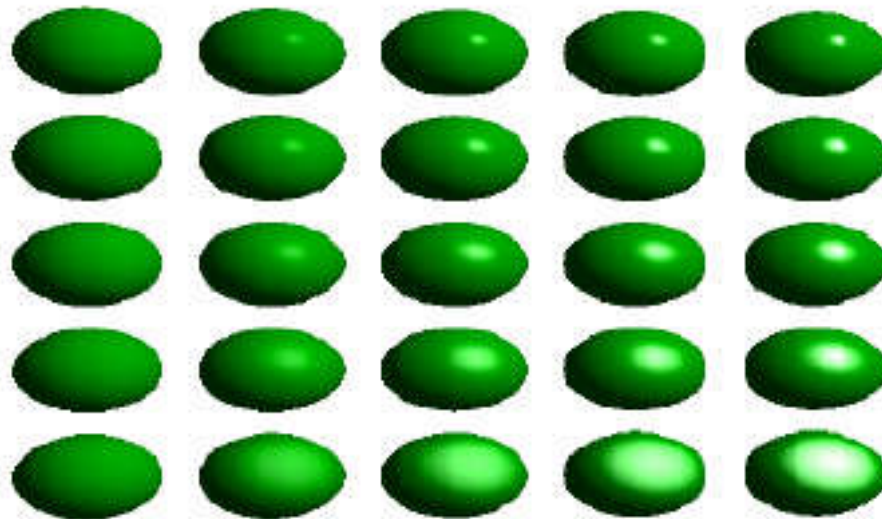
$(1 \leq n < \infty)$, 1: mate, ∞ : espejo)

$-\pi/2$

0

$\pi/2$

Ejemplo iluminación Phong



Modelo de iluminación de Phong, de izquierda a derecha el coeficiente
De reflexión especular de 0 a 1 y de arriba abajo el coeficiente
 n varía de 0 a 50.

Resumen

Modelo	Depende de la Normal	Depende del Observador	Ejemplo
Ambiente	NO	NO	
Difusa	SI	NO	
Especular	SI	SI	

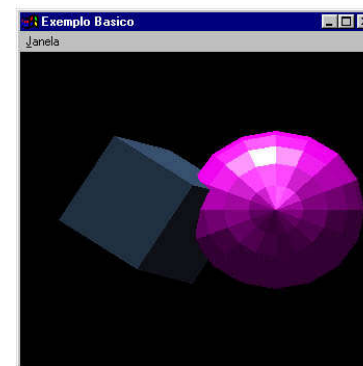
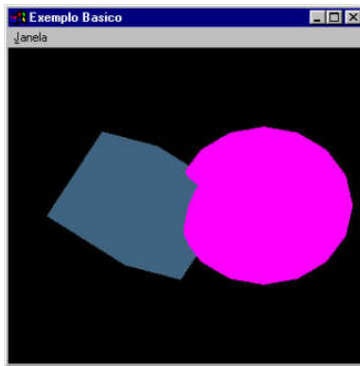
Modelo de iluminación

- Modelo global con las tres iluminaciones

$$I = I_a \cdot K_a + I_L \cdot \left(K_d \cdot (\vec{N} \cdot \vec{L}) + K_s \cdot (\vec{R} \cdot \vec{V})^n \right)$$

- Otros Efectos

- **Atenuación**: Consiste en no desestimar la atenuación de la luz con la distancia al observador.
- **Múltiples fuentes**: Hemos considerado siempre una única fuente de luz. Para tener varias en cuenta sólo hay que sumar sus efectos.
- **Color**: El estudio realizado es monocromo, tener en cuenta los tres colores implicará triplicar las expresiones



Iluminación en color

– Color:

$$I_R = I_{aR} \cdot K_a \cdot C_{dR} + I_{LR} \cdot f_{at} \cdot \left(K_d \cdot C_{dR} \cdot (\vec{N} \bullet \vec{L}) + K_S \cdot C_{eR} \cdot (\vec{R} \bullet \vec{V})^n \right)$$

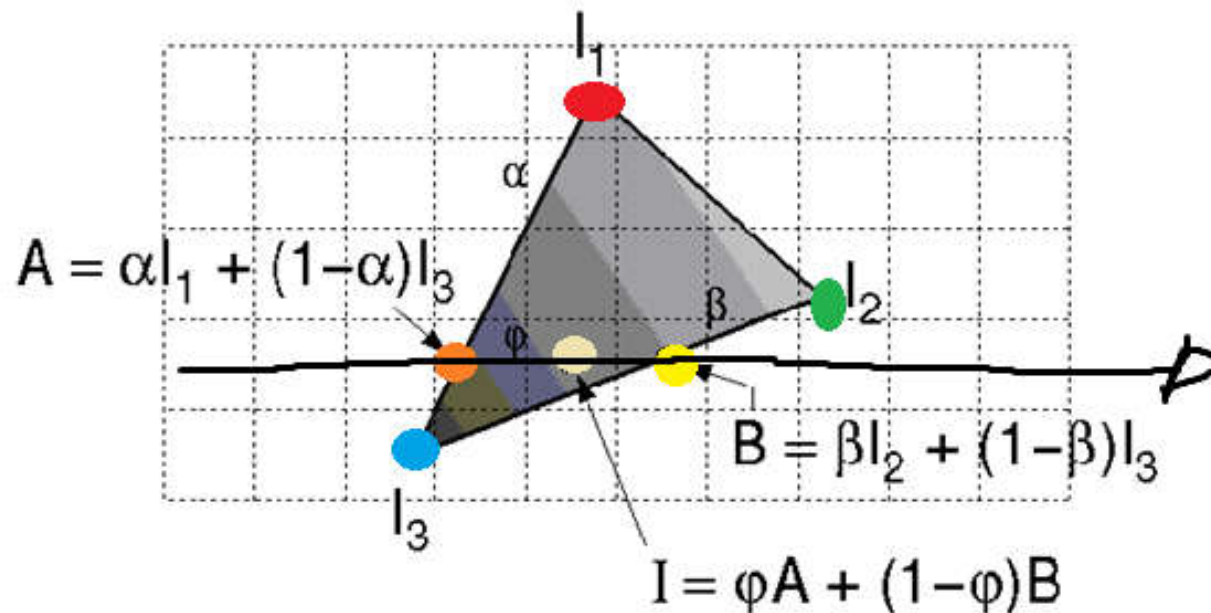
$$I_G = I_{aG} \cdot K_a \cdot C_{dG} + I_{LG} \cdot f_{at} \cdot \left(K_d \cdot C_{dG} \cdot (\vec{N} \bullet \vec{L}) + K_S \cdot C_{eG} \cdot (\vec{R} \bullet \vec{V})^n \right)$$

$$I_B = I_{aB} \cdot K_a \cdot C_{dB} + I_{LB} \cdot f_{at} \cdot \left(K_d \cdot C_{dB} \cdot (\vec{N} \bullet \vec{L}) + K_S \cdot C_{eB} \cdot (\vec{R} \bullet \vec{V})^n \right)$$

Gouraud shading (rastering)

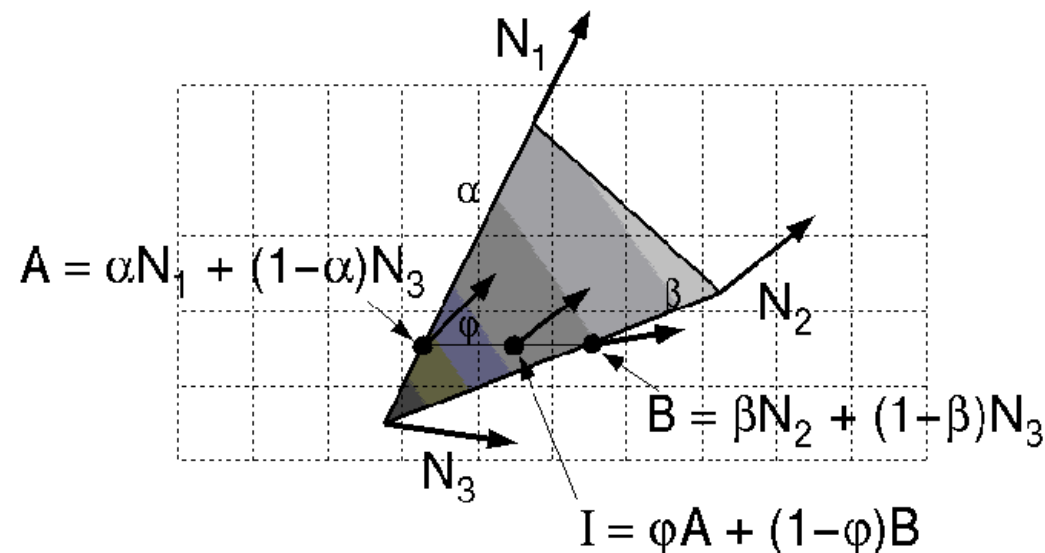
In Gouraud shading, la normal media es calculada para computar el color de cada vértice.

El color de cada punto del polígono es calculado mediante la **interpolación bilinear** entre los **colores** del mismo



PHONG shading (rastering)

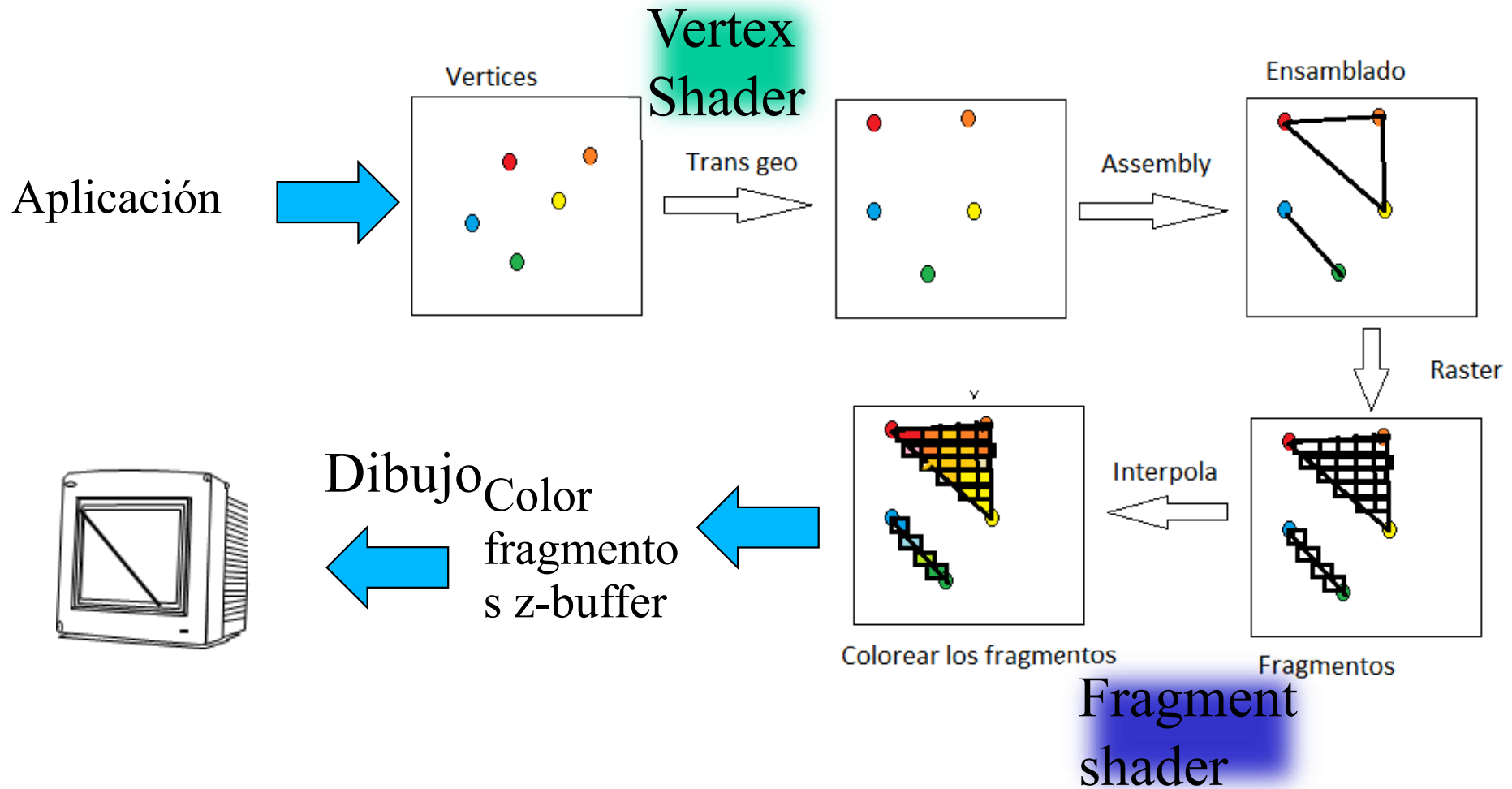
- Sombreado PHONG.
 - Mejor que los anteriores, pero consume más recursos máquina.
 - Se utiliza una interpolación bi-lineal para calcular la normal de cada punto del polígono.
 - A cada punto del polígono se le asocia una normal distinta con lo cual su color asociado será muy cercano a la realidad.
 - OpenGL no implementa PHONG directamente.
 - El programador es el encargado de interpolar las normales y asociarlas a todos los puntos.



Tutorial.

Iluminación OpenGL 3.3

Pipe 3.3



Iluminación básico

- El color de un objeto es fruto del color del objeto multiplicado por el color de la luz con la que se ilumina.
- Definiremos dos variables que pasaremos al shader.frag en el incluimos la multiplicación por de cada uno de ellos el en mismo.
- Correspondería a la luz básica de la versión 1.2

```
version 330 core
uniform vec3 objectColor;
uniform vec3 lightColor;
void main() {
    Gl_FragColor = vec4(lightColor * objectColor, 1.0); }
```

Luz Ambiente $I = I_a \cdot K_a$

```
//El color del objeto
unsigned int colorLoc = glGetUniformLocation(shaderProgram, "objectColor");
glUniform3f(colorLoc, 0.9f, 0.3f, 0.3f);
//El color de la luz
unsigned int lightLoc = glGetUniformLocation(shaderProgram, "lightColor");
glUniform3f(lightLoc, 0.9f, 0.6f, 0.6f);
```

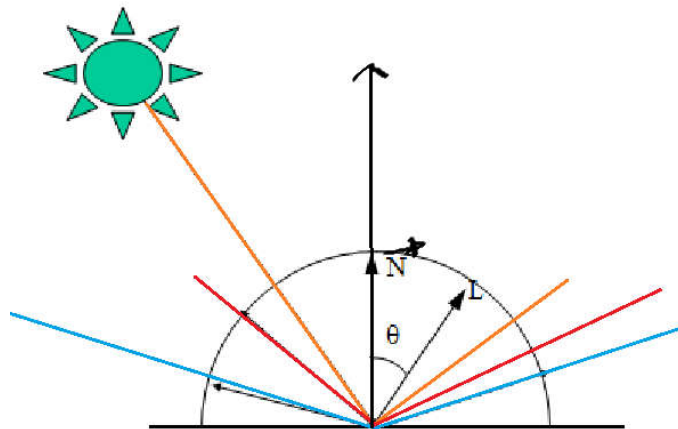
```
uniform vec3 lightColor;
uniform vec3 objectColor;
void main()
{
    float ambientI = 0.1;
    vec3 ambient = ambientI * lightColor;
    vec3 result = ambient * objectColor;
    gl_FragColor = vec4(result, 1.0);
}
```

Luz difusa.

- Tenemos las mismas luces, ambiente, difusa y especular con las mismas características.

$$I = I_L \cdot K_d \cos(\theta) = I_L \cdot K_d \cdot (\vec{N} \bullet \vec{L})$$

- Necesitamos en este caso.
- La atenuación.
- Las normales de las caras.
- La posición de la luz y el punto de cálculo para ver la dirección de la luz L.
- Estos parámetros debemos pasárselos al shader.



OpenGL 33

La carga es igual, pero tenemos que:

1) Especificar las coordenadas de las normales.

```
float vertices[] = {  
    //Frontal vertices                                Normales  
    -0.5f, -0.5f,  0.5f,                               0.0f, 0.0f, 1.0f,  
    0.5f, -0.5f,  0.5f,                               0.0f, 0.0f, 1.0f,  
    0.5f,  0.5f,  0.5f,                               0.0f, 0.0f, 1.0f,  
    0.5f,  0.5f,  0.5f,                               0.0f, 0.0f, 1.0f,  
    -0.5f,  0.5f,  0.5f,                               0.0f, 0.0f, 1.0f,  
    -0.5f, -0.5f, 0.5f,                               0.0f, 0.0f, 1.0f, .....  
    // vertices  
    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (void*)0);  
    glEnableVertexAttribArray(0);  
    // Normales  
    glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (void*)(3 *  
    sizeof(float)));  
    glEnableVertexAttribArray(1);
```

Luz difusa. *Vertex.shader*

$$I = I_L \cdot K_d \cos(\theta) = I_L \cdot K_d \cdot (\vec{N} \bullet \vec{L})$$

```
#version 330 core
layout (location = 0) in vec3 aPos;
layout (location = 1) in vec3 aNormal;
out vec3 Normal;
out vec3 FragPos;
uniform mat4 model;
uniform mat4 view;
uniform mat4 projection;

void main()
{
    gl_Position = projection * view * model * vec4(aPos, 1.0f);
    Normal = mat3(transpose(inverse(model))) * aNormal;
    FragPos = vec3(model * vec4(aPos, 1.0));
}
```

Posición del fragmento de trabajo en las coordenadas de la scena

Luz difusa. *Shader.frag*

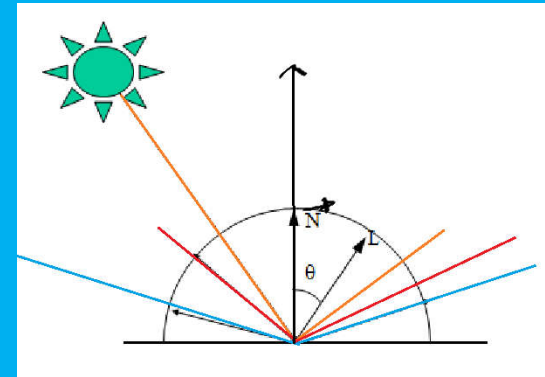
$$I = I_L \cdot K_d \cos(\theta) = I_L \cdot K_d \cdot (\vec{N} \bullet \vec{L})$$

```
in vec3 Normal;
in vec3 FragPos;

uniform vec3 lightPos;
uniform vec3 lightColor;
uniform vec3 objectColor;
void main()
{
    // ambient
    float ambientI = 0.5;
    vec3 ambient = ambientI * lightColor;

    // diffuse
    vec3 norm = normalize(Normal);    //Normaliación
    vec3 lightDir = normalize(lightPos - FragPos); //rayo luz fragmento
    float diff = max(dot(norm, lightDir), 0.0);    //producto escalar
    vec3 diffuse = diff * lightColor;

    vec3 result = (ambient + diffuse) * objectColor;
    FragColor = vec4(result, 1.0);
}
```



Luz difusa.

```
glUseProgram(shaderProgram); //

//El color de la luz
unsigned int lightColorLoc = glGetUniformLocation(shaderProgram, "lightColor");
glUniform3f(lightColorLoc, 0.9f, 0.6f, 0.6f);

// la posicion de la luz lo hago abajo porque va con el objeto de que se mueve
float lightPosX = 0.0 + -5 * sin((float)glfwGetTime() / 2.0f);
float lightPosY = 0.0;
float lightPosZ = 0.0 + (-5) * cos((float)glfwGetTime() / 2.0f);

unsigned int lightPosLoc = glGetUniformLocation(shaderProgram, "lightPos");
glUniform3f(lightPosLoc, lightPosX, lightPosY, lightPosZ);
```

La luz especular

- Modelo de iluminación Especular:

$$I = I_L \cdot K_S \cos^n(\alpha) = I_L \cdot K_S \cdot (\vec{R} \cdot \vec{V})^n$$

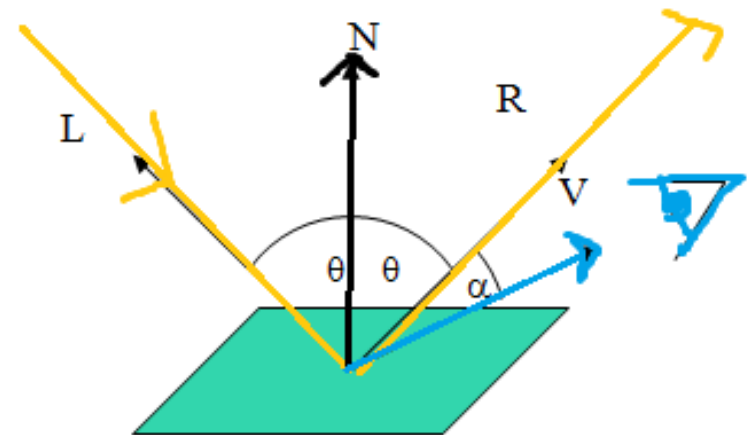
α : Ángulo entre $\vec{R}$ y $\vec{V}$. Desviación del ángulo de reflexión ideal

K_S : Coeficiente de reflexión especular dependiendo del material del objeto $[0,1]$

n : Coeficiente de especularidad o concentración del brillo.

($1 \leq n < \infty$, 1: mate, ∞ : espejo)

Lo único que me falta es la posición d
usuario/cámara.



```
#version 330 core
out vec4 FragColor;
```

```
in vec3 Normal;
in vec3 FragPos;
```

```
uniform vec3 viewPos;
```

```
uniform vec3 lightPos;
uniform vec3 lightColor;
uniform vec3 objectColor;
void main()
```

```
{
    // ambient
    float ambientI = 0.5;
    vec3 ambient = ambientI * lightColor;
```

```
    // diffuse
    vec3 norm = normalize(Normal);
    vec3 lightDir = normalize(lightPos - FragPos);
    float diff = max(dot(norm, lightDir), 0.0);
    vec3 diffuse = diff * lightColor;
```

```
    //Speculas
```

```
    float specularStrength = 1.0;
    vec3 viewDir = normalize(viewPos - FragPos);
    vec3 reflectDir = reflect(-lightDir, norm);
    float spec = pow(max(dot(viewDir, reflectDir), 0.0), 128); //Potencia
    vec3 specular = specularStrength * spec * lightColor;
```

```
    vec3 result = (ambient + diffuse + specular) * objectColor;
    FragColor = vec4(result, 1.0);
```

```
}
```

Luz Specular. Fragment.Shader

//la posicion del usuario/camara (0,0,10) en nuestro caso

```
unsigned int viewPosLoc = glGetUniformLocation(shaderProgram, "viewPos");
glUniform3f(viewPosLoc, 0.0f, 0.0f, 10.0f);
```

$$I = I_L \cdot K_S \cos^n(\alpha) = I_L \cdot K_S \cdot (\vec{R} \cdot \vec{V})^n$$

//Usuario Fragmento

//Reflejo de la luz

Temario

Tema 1: Introducción.

Introducción *100%*

Revisión histórica da computación gráfica. *Ver documentación 100%.*

Tema 2: Estándares gráficos Evolución histórica, OpenGL *100%*

Tema 3: Formas 2D e Antialiasing (final del curso) *80%*

Tema 4: Transformacións xeométricas, 2D, 3D. *100 %*

Tema 5: Proyeccións, modelo de cámara sintética. *100%*

Tema 6: Modelado e texturas *50%*

Tema 7: Color, Iluminación y sombreado *100%*

Tema 8: Determinación de superficies visibles, z-Buffer *50%*

Temario

Tema 1: Introducción.

Introducción *100%*

Revisión histórica da computación gráfica. *Ver documentación 100%.*

Tema 2: Estándares gráficos Evolución histórica, OpenGL *40%*

Tema 3: Formas 2D e Antialiasing (final del curso) *100%*

Tema 4: Transformacións xeométricas, 2D, 3D. *100 %*

Tema 5: Proyeccións, modelo de cámara sintética. *100%*

Tema 6: Modelado e texturas *50%*

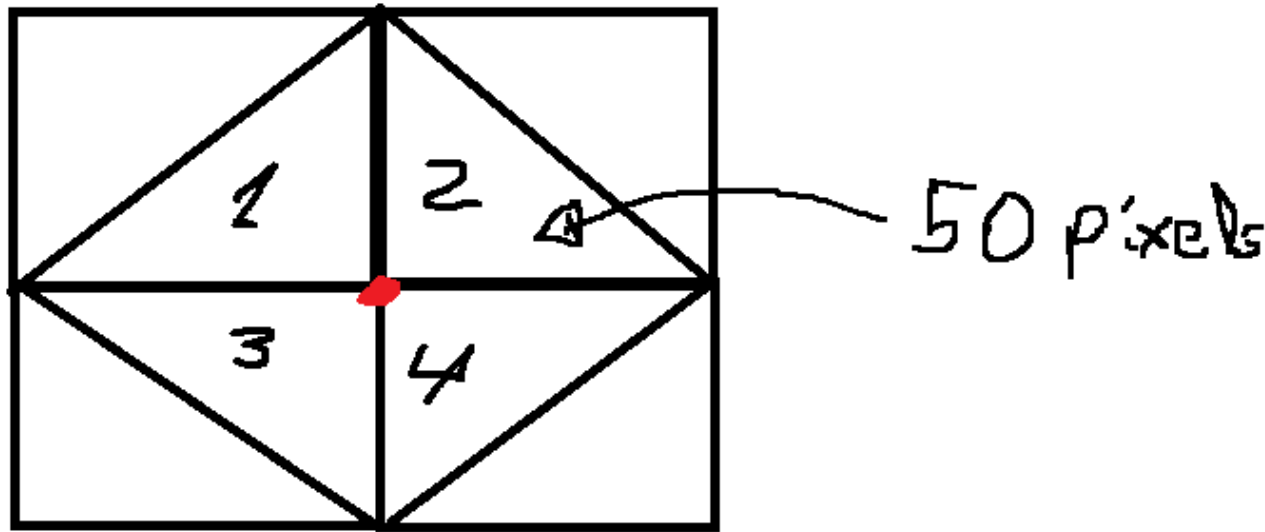
Tema 7: Color, Iluminación y sombreado **100%**

Tema 8: Determinación de superficies visibles, z-Buffer *50%*

Tema 9: Shaders GLSL

Problema

Por ejemplo, +/-la misma cantidad de tiempo para calcular la iluminación en un vértice como lo hace por píxel. Calcular entonces cuantas evaluaciones tenemos que hacer para el sombreado de Sombreado de Phong y Gouraud. Tenemos un objeto teselado, cada vértice lo comparten 4 triángulos. En promedio, cada triángulo cubre 50 pixeles. Supongamos que tenemos que evaluar 100 triángulos.



Problema normalización.

glEnable(GL_NORMALIZE)

Supongamos una luz lambert $I = 1$ de dirección $(0, -1, 0)$ que incide sobre la cara formada por los vértices $(0, 0, 0)$ $(1, 0, 0)$ $(0, 0, 1)$. ¿Cuál es la Intensidad de luz resultante? ¿Qué pasa si la cara es $(0, 0, 0)$ $(2, 0, 0)$ $(0, 0, 2)$?. Razona la respuesta.

$$I = I_L \cdot K_d \cos(\theta) = I_L \cdot K_d \cdot (\vec{N} \bullet \vec{L})$$

Problema normalización.

glEnable(GL_NORMALIZE)

Supongamos una luz lambert $I = 1$ de dirección $(0, -1, 0)$ que incide sobre la cara formada por los vértices $(0, 0, 0)$ $(1, 0, 0)$ $(0, 0, 1)$. ¿Cuál es la Intensidad de luz resultante? ¿Qué pasa si la cara es $(0, 0, 0)$ $(2, 0, 0)$ $(0, 0, 2)$?. Razona la respuesta.

$$I = I_L \cdot K_d \cos(\theta) = I_L \cdot K_d \cdot (\vec{N} \bullet \vec{L})$$